RV1106 RV1103 Quick Start

ID: RK-JC-YF-415

Release Version: V2.5.0

Release Date: 2026-09-30

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD. (“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY.

THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO PRODUCT UPGRADES OR OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", and "瑞芯" are registered trademarks of Rockchip and are owned by Rockchip.

All other registered trademarks or trademarks mentioned in this document are owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, Software Boulevard, Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document is intended for users who are using the RV1103B/RV1103/RV1106B/RV1106 RKNN SDK for the first time. It provides the shortest path to complete development board connection, RKNN-Toolkit2 environment setup, model conversion, on-board execution, and first inference verification. The examples used in this document are integrated into the RKNN Model Zoo.

Intended Audience

This document is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Version	Modifier	Date	Modify Description	Reviewer
V1.6.0	HPC	2023-11-28	Initial version	Vincent
V2.0.0-beta0	HPC	2024-03-15	Update version number	Vincent
V2.1.0	HPC	2024-08-01	Add support for RV1103B	Vincent
V2.2.0	HPC	2024-09-04	Add support for RV1106B	Vincent
V2.3.0	HPC	2024-11-06	Add support for installing RKNN-Toolkit2 via pip source	Vincent
V2.3.2	HPC	2025-04-03	Update version number	Vincent
V2.4.2	HPC	2026-03-19	Update version number	Vincent
V2.5.0	HPC	2026-09-30	Fix document	Vincent

Table of Contents

RV1106 RV1103 Quick Start

1. Before You Begin

    1.1 Goals of This Guide

    1.2 Understand the RKNN SDK Deployment Flow

    1.3 Supported Platforms and Version Requirements

    1.4 Select the Example Type

    1.5 Obtain the SDK and Related Repositories

2. Hardware Connection and Check

    2.1 Hardware Checklist

    2.2 Introduction to Development Board and Connection Tools

    2.3 Connect the Development Board

    2.4 Check the Device Connection Status

3. Host and Board Environment Setup

    3.1 Install RKNN-Toolkit2

        3.1.1 Install the Python Environment

        3.1.2 Install RKNN-Toolkit2

    3.2 Check the Toolkit Installation

    3.3 Install or Check the Board-Side RKNPU2 Environment

        3.3.1 Check the RKNPU2 Driver Version

        3.3.2 Check RKNPU2 Mini Runtime and Server

        3.3.3 Install or Update the RKNPU2 Environment

    3.4 Install Compilation Tools

        3.4.1 Install CMake

        3.4.2 Install the Compiler

            3.4.2.1 Confirm the Development Board System Architecture

            3.4.2.2 Install the GCC Cross-Compiler

    3.5 Runtime Environment Pre-Check

4. Run the First General Model

    4.1 Obtain the Example and Model

    4.2 Convert the Model

    4.3 Run the RKNN Python Demo

    4.4 Compile and Deploy the RKNN C Demo

        4.4.1 Compilation

        4.4.2 Push Files to the Board

4.5 Run on the Board and Verify the Output

4.6 Dataset Accuracy Evaluation (Optional)

5. Run the RKNN Python Demo in Docker (Optional)

5.1 Install Docker

5.2 Prepare the RKNN-Toolkit2 Docker Image

5.2.1 Build an Image from a Dockerfile

5.2.2 Load a Packaged Image

5.2.3 Check the Image

5.3 Run the RKNN Python Demo in the Container

6. Common First-Run Issues and Next Steps

6.1 Device Cannot Be Recognized

6.2 rknn_server Does Not Start Correctly

6.3 Insufficient Memory Causes Model Loading Failure

6.4 Next-Step Documentation

1. Before You Begin

This guide provides the shortest path to complete the first RKNN SDK run on RV1103/RV1103B/RV1106/RV1106B, including hardware connection, environment checks, model conversion, on-board debugging, and C/C++ deployment on the board.

> **Note:** For the complete development flow, model conversion and quantization, environment installation, and systematic troubleshooting, refer to *RKNN2_SDK_Development_Guide*. For RKNN-Toolkit2 Python APIs, refer to *RKNN2_Toolkit_Python_API_Reference*. For RKNN-Toolkit-Lite2 Python APIs, refer to *RKNN2_Toolkit_Lite_Python_API_Reference*. For board-side C APIs and RKNPU2 Mini Runtime (`librknnmrt.so`) related interfaces, refer to *RKNN2_Runtime_C_API_Reference*.

1.1 Goals of This Guide

After completing this guide, you should be able to:

1. Confirm that the development host and development board are connected correctly.
2. Install and verify the RKNN-Toolkit2 development environment.
3. Check or update the board-side RKNPU2 Mini Runtime (`librknnmrt.so`) and `rknn_server` .
4. Use RKNN Model Zoo to download the YOLOv5 model and convert it to an RKNN model.
5. Complete one on-board inference using the RKNN Python Demo.
6. Compile and run the RKNN C Demo on the board and verify the output.

1.2 Understand the RKNN SDK Deployment Flow

The first run on RV1103/RV1103B/RV1106/RV1106B mainly involves development tools on the computer and the runtime environment on the development board.

Component	Location	Description
RKNN-Toolkit2	Computer	Converts original models such as ONNX models to RKNN models and supports on-board debugging
RKNN Model Zoo	Computer	Provides model conversion examples, Python Demos, and C/C++ Demos
`rknn_server`	Development board	Handles on-board inference requests sent by the computer through USB
RKNPU2 Mini Runtime (`librknnmrt.so`)	Development board	Loads RKNN models and invokes the NPU to perform inference

The differences between RKNPU2 Mini Runtime and RKNPU2 Runtime are as follows:

Item	RKNPU2 Mini Runtime	RKNPU2 Runtime
Runtime library	`librknnmrt.so`	`librknnrt.so`
Positioning	A streamlined Runtime for resource-constrained platforms such as RV1103/RV1103B/RV1106/RV1106B	Standard RKNPU2 Runtime
Applicability to this guide	**Applicable. All board-side runtime references later in this guide refer to `librknnmrt.so`.**	**Not applicable to the examples in this guide. Do not replace `librknnmrt.so` with `librknnrt.so`.**

> **Note:** In the remainder of this guide, **RKNPU2 Mini Runtime** refers specifically to the board-side `librknnmrt.so`. Except for this section, which distinguishes the two Runtime variants, this guide does not use the generic term "RKNPU2 Runtime" for the board-side runtime library, to avoid confusion with `librknnrt.so`.

The recommended first-run path is:

1. Connect the development board and confirm that it is visible in `adb devices`.
2. Install RKNN-Toolkit2 and complete the import check.
3. Check the RKNPU2 driver, Mini Runtime, and `rknn_server`.
4. Download the YOLOv5 example model and convert it to RKNN format.
5. Run the RKNN Python Demo first to verify model conversion and the on-board debugging path.
6. Compile, push, and run the RKNN C Demo to complete board-side deployment verification.

1.3 Supported Platforms and Version Requirements

This guide supports the following platforms:

Platform	Support
RV1103	Supported
RV1103B	Supported
RV1106	Supported
RV1106B	Supported

The development host examples are based on Ubuntu 18.04 / Ubuntu 20.04 / Ubuntu 22.04. Python 3.8 is recommended for the RKNN-Toolkit2 environment.

> **Note:** RKNN models generated for RV1103/RV1106 cannot be shared with RV1103B/RV1106B. Select the correct platform parameter according to the actual development board during model conversion.

1.4 Select the Example Type

This guide uses the YOLOv5 example in RKNN Model Zoo throughout the first-run flow.

Example	Purpose	Model Zoo Directory
YOLOv5 Python Demo	Model conversion and on-board inference verification	`examples/yolov5/python`
YOLOv5 C Demo	C/C++ compilation and board-side deployment verification	`examples/yolov5/cpp`

The Python Demo and C Demo use the same YOLOv5 ONNX model and the same converted RKNN model, so the model only needs to be downloaded and converted once.

1.5 Obtain the SDK and Related Repositories

It is recommended to create a dedicated directory for RKNN-related repositories. For example, create a folder named `Projects` and place RKNN-Toolkit2 and RKNN Model Zoo in this directory.

```
# Create the Projects folder
mkdir Projects

# Switch to this directory
cd Projects

# Download the RKNN-Toolkit2 repository
git clone https://github.com/airockchip/rknn-toolkit2.git --depth 1

# Download the RKNN Model Zoo repository
git clone https://github.com/airockchip/rknn_model_zoo.git --depth 1
```

> **Note:** `--depth 1` means that only the latest commit is cloned. If `git clone` fails, you can also download the compressed package directly from GitHub and extract it into this directory.

The default directory structure used in the following examples is:

```
Projects
├── rknn-toolkit2
│   ├── doc
│   ├── rknn-toolkit2
│   │   ├── packages
│   │   ├── docker
│   │   └── ...
│   ├── rknpu2
│   │   ├── runtime
│   │   └── ...
│   └── ...
└── rknn_model_zoo
    ├── datasets
    ├── examples
    └── ...
```

2. Hardware Connection and Check

This chapter covers only the hardware connection and device visibility checks required for the first run. After confirming that the development board can be recognized by the computer through ADB, continue with environment installation and model conversion.

2.1 Hardware Checklist

The following hardware is required to run the examples in this guide:

Hardware	Quantity
Ubuntu computer	1
RV1103/RV1103B/RV1106/RV1106B development board	1
Micro USB data cable	1
12V-2A power adapter	1

2.2 Introduction to Development Board and Connection Tools

RV1103 Development Board

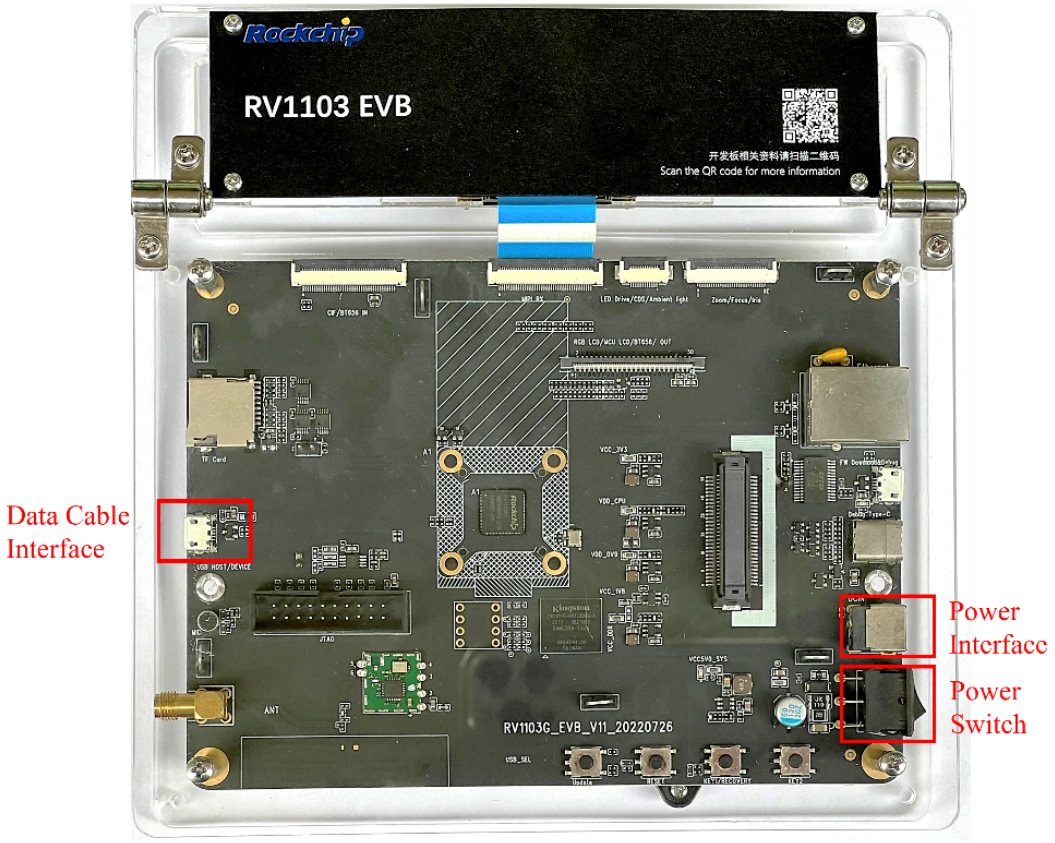

Figure 2-1 RV1103 development board

RV1103B Development Board

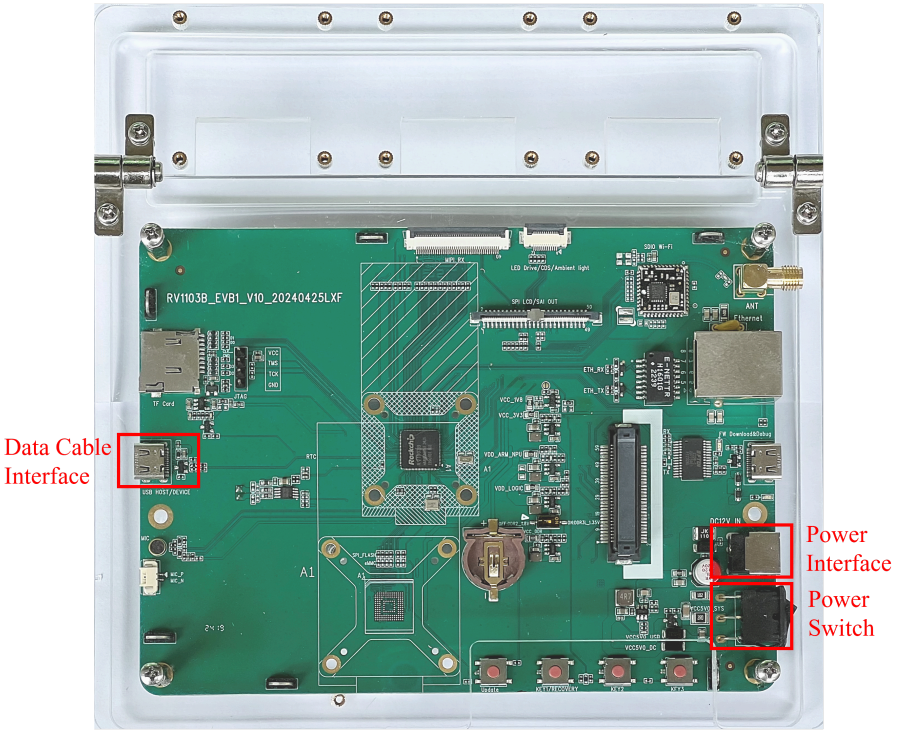

Figure 2-2 RV1103B development board

RV1106 Development Board

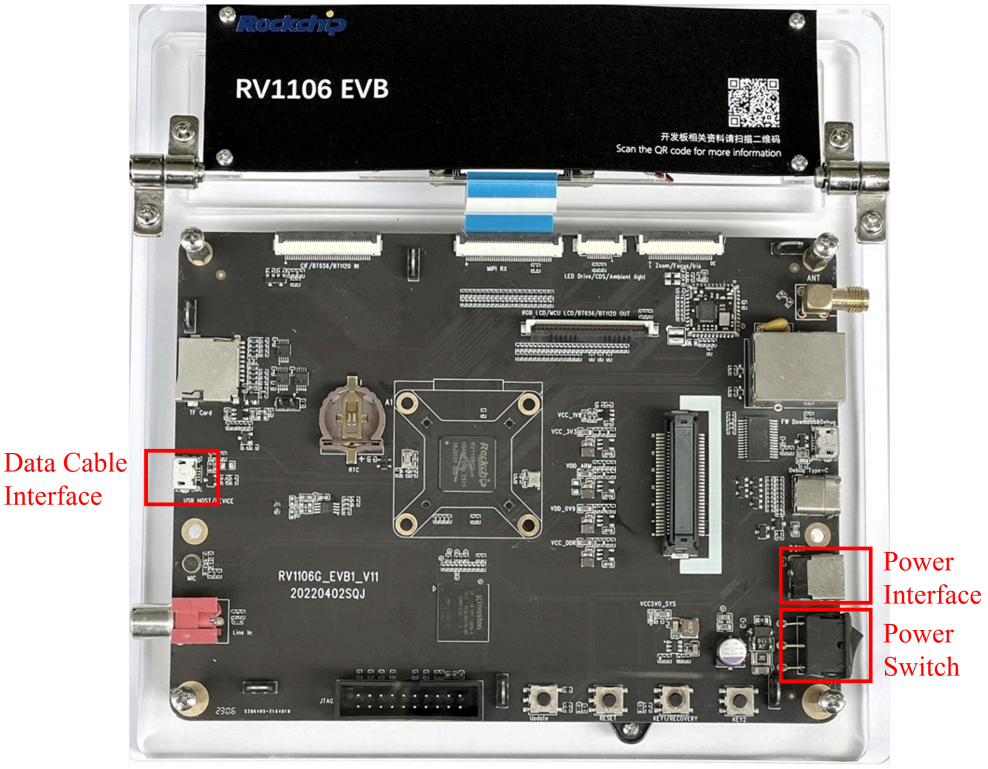

Figure 2-3 RV1106 development board

Micro USB Data Cable

Figure 2-4 Micro USB data cable

Power Adapter

Figure 2-5 Power adapter with 12V-2A output

2.3 Connect the Development Board

The following steps use RV1106 as an example to illustrate how to connect the development board to the computer:

1. Prepare a computer running Ubuntu 18.04 / Ubuntu 20.04 / Ubuntu 22.04.
2. Locate the power connector on the development board and connect the power adapter.
3. Use a data cable to connect the development board to the computer. Due to differences between production batches, the type and location of the data cable connector may vary. Generally, the connector marked `OTG` on the board is the data cable connector.
4. Turn on the power switch and wait for the development board system to start.

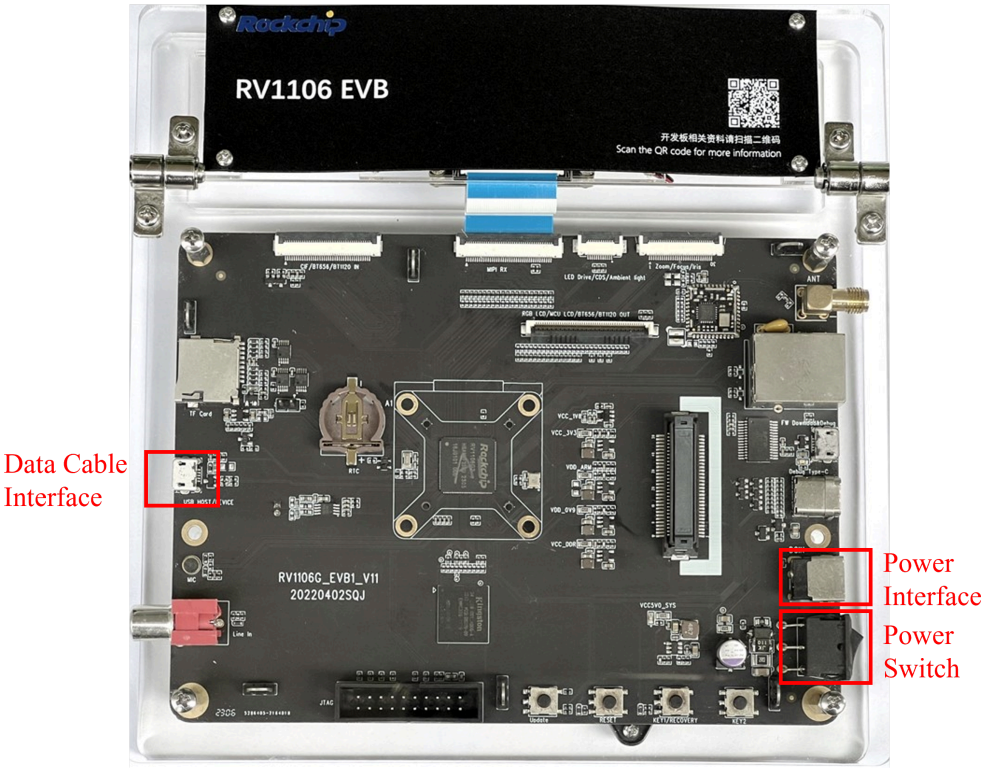

Figure 2-6 RV1106 development board

2.4 Check the Device Connection Status

Run the following commands in the computer terminal:

```
# Install adb if it is not already installed
sudo apt install adb

# Query ADB devices
adb devices
```

When the connection is successful, the output is similar to the following. `421ca310c9b9242b` is the RV1106 device ID in this example; the actual device ID depends on your board.

```
List of devices attached
421ca310c9b9242b    device
```

Before continuing with installation and model conversion, confirm at least the following checks:

Check Item	Command	Expected Result
ADB is available	`adb version`	ADB version information is displayed correctly
Development board is visible	`adb devices`	The board appears in the list with the status `device`

If no device is displayed by `adb devices`, refer to 6.1 Device Cannot Be Recognized.

3. Host and Board Environment Setup

This chapter prepares RKNN-Toolkit2, RKNPU2, and compilation tools required for the first run. The main flow uses direct installation on the computer. If you want to use Docker, refer to Chapter 5.

3.1 Install RKNN-Toolkit2

3.1.1 Install the Python Environment

If Python 3.8 (recommended) is not installed on the system, or if multiple Python environments exist, it is recommended to use Miniforge to create an independent Python 3.8 environment.

Step 1: Install Miniforge

First, check whether Miniforge is already installed. If it is installed, skip this step.

```
conda -V
# Reference output: conda 23.3.1, indicating that the Miniforge conda version is
23.3.1
# If 'conda: command not found' is displayed, Miniforge is not installed
```

If Miniforge is not installed, download the installer:

```
wget -c https://github.com/conda-
forge/miniforge/releases/latest/download/Miniforge3-Linux-x86_64.sh
```

Then run:

```
chmod +x Miniforge3-Linux-x86_64.sh
bash Miniforge3-Linux-x86_64.sh
```

Step 2: Create the Python Environment

Enter the Miniforge base environment:

```
source ~/miniforge3/bin/activate
# Upon successful activation, the command prompt changes to:
# (base) xxx@xxx:~$
```

Create a Python 3.8 environment named `toolkit2`:

```
conda create -n toolkit2 python=3.8
```

Activate the `toolkit2` environment:

```
conda activate toolkit2
# Upon successful activation, the command prompt changes to:
# (toolkit2) xxx@xxx:~$
```

> **Note:** All subsequent Python-related commands are assumed to be executed in the `toolkit2` environment.

3.1.2 Install RKNN-Toolkit2

After activating the `toolkit2` environment, RKNN-Toolkit2 can be installed either from the pip source or from a local wheel package.

Method 1: Install from the pip Source

```
pip install rknn-toolkit2 -i https://pypi.org/simple

# If RKNN-Toolkit2 is already installed, upgrade it using:
pip install rknn-toolkit2 -i https://pypi.org/simple --upgrade
```

Method 2: Install from a Local Wheel Package

```
# Switch to the rknn-toolkit2 directory
cd Projects/rknn-toolkit2/rknn-toolkit2

# Select the requirements file according to the Python version and processor
architecture
# cpxx is the Python version number
pip install -r packages/x86_64/requirements_cpxx.txt
# pip install -r packages/arm64/arm64_requirements_cpxx.txt

# Install RKNN-Toolkit2
# x.x.x is the RKNN-Toolkit2 version number, and cpxx is the Python version number
pip install packages/x86_64/rknn_toolkit2-x.x.x-cpxx-cpxx-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl
# pip install packages/arm64/rknn_toolkit2-x.x.x-cpxx-cpxx-
manylinux_2_17_aarch64.manylinux2014_aarch64.whl
```

3.2 Check the Toolkit Installation

Keep the `toolkit2` environment activated and enter Python interactive mode:

```
python
```

Then import the `RKNN` class:

```
from rknn.api import RKNN
```

If no error is reported during the import, the basic RKNN-Toolkit2 installation check has passed.

If the installation fails, refer to the RKNN-Toolkit2 installation and troubleshooting sections in *RKNN2_SDK_Development_Guide*.

3.3 Install or Check the Board-Side RKNPU2 Environment

For convenience, the remainder of this document uses "board" to refer to the development board.

The on-board debugging feature of RKNN-Toolkit2 requires the RKNPU2 environment to be installed on the board and `rknn_server` to be running.

Component	Description
RKNN Server (`rknn_server`)	A background proxy service running on the development board. It processes data sent by the computer through USB and invokes the board-side Mini Runtime to perform inference
RKNPU2 Mini Runtime (`librknnmrt.so`)	Loads RKNN models in the system and invokes the NPU to perform inference

3.3.1 Check the RKNPU2 Driver Version

Run the following command on the board to query the RKNPU2 driver version:

```
dmesg | grep -i rknpu
```

As shown in the following figure, the RKNPU2 driver version in this example is 0.8.5:

```
# dmesg | grep -i rknpu
[    4.431371] RKNPU ff660000.npu: RKNPU: rknpu iommu device-tree entry not found!, using non-iommu mode
[    4.432210] RKNPU ff660000.npu: RKNPU: Initialized RKNPU driver: v0.8.5 for 20230202
[    4.432314] RKNPU ff660000.npu: dev_pm_opp_set_regulators: no regulator (rknpu) found: -19
```

Figure 3-1 RKNPU2 driver version information

Official Rockchip development board firmware includes the RKNPU2 driver. If the command above cannot query the NPU driver version, third-party firmware without the NPU driver may be

in use. If firmware source code is available, set `CONFIG_ROCKCHIP_RKNPU` to `y` in the kernel config, rebuild the kernel driver, and flash it.

> **Recommendation:** RKNPU2 driver version >= 0.9.2.

3.3.2 Check RKNPU2 Mini Runtime and Server

If RKNN Server or the Mini Runtime library is not installed on the board, or if their versions are inconsistent, reinstall the RKNPU2 environment.

> **Note:**
>
> 1. If an RKNN model with dynamic input dimensions is used, the RKNN Server and Mini Runtime library versions must be >= 1.5.0.
> 2. The versions of `rknn_server`, the Mini Runtime library, and RKNN-Toolkit2 should be consistent. It is recommended to install components from the same latest release.

Step 1: Check Whether `rknn_server` Can Be Started

```
# Switch to the board
adb shell

# Start rknn_server
restart_rknn.sh
```

If output similar to the following is displayed, `rknn_server` has started successfully:

```
start rknn server, version: x.x.x
```

Step 2: Check the Server and Mini Runtime Versions

```
# Query the rknn_server version
strings /oem/usr/bin/rknn_server | grep -i "rknn_server version"

# Query the librknnmrt.so version
strings /oem/usr/lib/librknnmrt.so | grep -i "librknnmrt version"
```

Reference output:

```
rknn_server version: x.x.x
librknnmrt version: x.x.x
```

3.3.3 Install or Update the RKNPU2 Environment

If a version-consistent RKNPU2 environment is already installed, skip this section.

Switch to the `rknpu2` directory, use ADB to push the corresponding `rknn_server` and `librknnmrt.so` to the board, and then start `rknn_server`:

```
# Switch to the rknpu2 directory
cd Projects/rknn-toolkit2/rknpu2

# Push rknn_server to the board
adb push runtime/Linux/rknn_server/armhf-uclibc/usr/bin/* /oem/usr/bin

# Push librknnmrt.so
adb push runtime/Linux/librknn_api/armhf-uclibc/librknnmrt.so /oem/usr/lib

# Switch to the board
adb shell

# Grant executable permissions
chmod +x /oem/usr/bin/rknn_server
chmod +x /oem/usr/bin/start_rknn.sh
chmod +x /oem/usr/bin/restart_rknn.sh

# Restart the rknn_server service
restart_rknn.sh
```

3.4 Install Compilation Tools

3.4.1 Install CMake

Run the following commands in the computer terminal:

```
# Update package list
sudo apt update

# Install cmake
sudo apt install cmake
```

3.4.2 Install the Compiler

3.4.2.1 Confirm the Development Board System Architecture

Run the following command on the computer:

```
adb shell uname -a
```

The reference output is shown below. `armv7l` indicates the ARMv7 architecture, which belongs to the armhf architecture. This architecture type will be used later when compiling the RKNN C Demo.

```
Linux Rockchip 5.10.160 #1 Tue Oct 24 18:52:11 CST 2023 armv7l GNU/Linux
```

3.4.2.2 Install the GCC Cross-Compiler

GCC download link: https://console.zbox.filez.com/l/c1cKaR (Extraction code: `rknn`).

After downloading, extract the package. It is recommended to extract the GCC package into the `Projects` directory:

```
Projects
├── rknn-toolkit2
├── rknn_model_zoo
└── arm-rockchip831-linux-uclibcgnueabihf
```

The GCC compiler path used by the following examples is:

```
Projects/arm-rockchip831-linux-uclibcgnueabihf/bin/arm-rockchip831-linux-uclibcgnueabihf
```

3.5 Runtime Environment Pre-Check

Before model conversion, it is recommended to complete the following checks:

Check Item	Command or Location	Expected Result
Toolkit	Enter Python as described in Section 3.2 and execute `from rknn.api import RKNN`	No error
Model Zoo	`Projects/rknn_model_zoo/examples/yolov5`	Example directory exists
ADB	`adb devices`	Development board status is `device`
RKNPU2 driver	`adb shell "dmesg | grep -i rknpu"`	RKNPU2 driver information can be queried
RKNN Server	`adb shell restart_rknn.sh`	`rknn_server` starts normally
Mini Runtime version	Query the versions of `rknn_server` and `librknnmrt.so`	Versions are consistent
CMake	`cmake --version`	Version information is displayed correctly
GCC	Check `Projects/arm-rockchip831-linux-uclibcgnueabihf`	Cross-compiler directory exists

After the checks above pass, continue with the first model.

4. Run the First General Model

This chapter uses RV1106 and YOLOv5 as examples to complete model acquisition, RKNN model conversion, Python on-board inference, and C/C++ board-side deployment.

> **Note:** Parameters and usage may differ between Model Zoo examples. Follow the `README.md` in the corresponding example directory.

4.1 Obtain the Example and Model

RKNN Model Zoo provides example code for common models. The project directory structure is as follows:

```
rknn_model_zoo
├── 3rdparty          # Third-party libraries
├── datasets          # Datasets
├── examples          # Example code
├── utils             # Common utilities, such as file operations and drawing
├── build-android.sh  # Build script for Android development boards
├── build-linux.sh    # Build script for Linux development boards
└── ...
```

Taking YOLOv5 as an example:

```
rknn_model_zoo
├── examples
│   └── yolov5
│       ├── cpp       # C/C++ example code
│       ├── model     # Models, test images, and other files
│       ├── python    # Model conversion script and Python example code
│       └── README.md
└── ...
```

Switch to the `model` directory and run `download_model.sh` to download the YOLOv5 ONNX model:

```
cd Projects/rknn_model_zoo/examples/yolov5/model

# Download the yolov5 ONNX model
# Example download result: model/yolov5s_relu.onnx
./download_model.sh
```

4.2 Convert the Model

Switch to the YOLOv5 Python example directory and use `convert.py` to convert the ONNX model to an RKNN model:

```
cd Projects/rknn_model_zoo/examples/yolov5/python

# Usage: python convert.py model_path [rv1103|rv1103b|rv1106|rv1106b] [i8/fp]
[output_path]
python convert.py ../model/yolov5s_relu.onnx rv1106 i8 ../model/yolov5s_relu.rknn
```

> **Note:** RKNN models generated for RV1103/RV1106 cannot be shared with RV1103B/RV1106B. When using another development board, modify the platform parameter in the command accordingly.

4.3 Run the RKNN Python Demo

The RKNN Python Demo can run the converted YOLOv5 RKNN model on the board through on-board debugging:

```
cd Projects/rknn_model_zoo/examples/yolov5/python

# Usage: python yolov5.py --model_path {rknn_model} --target {target_platform} --
img_show
# --img_show displays the result image
python yolov5.py --model_path ../model/yolov5s_relu.rknn --target rv1106 --img_show
```

If you want to run the original ONNX model on the computer first, execute:

```
python yolov5.py --model_path ../model/yolov5s_relu.onnx --img_show
```

The default input image is `model/bus.jpg`. The output is shown below:

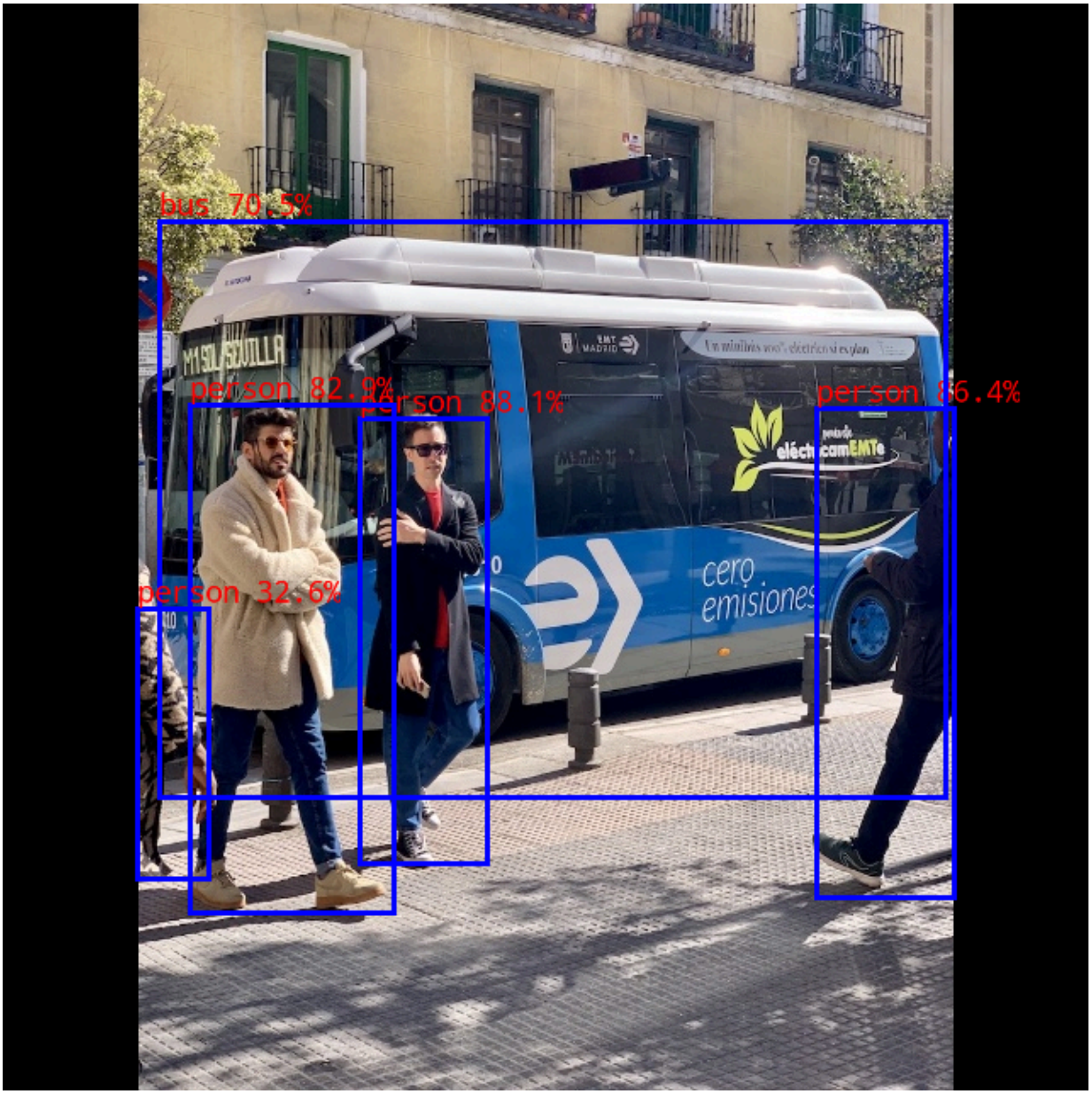

Figure 4-1 RKNN Python Demo output image

If the Python Demo completes inference successfully, the basic model conversion, ADB connection, `rknn_server`, and board-side Mini Runtime path has been verified.

4.4 Compile and Deploy the RKNN C Demo

The Python Demo and C Demo use the same converted RKNN model, so Sections 4.1 and 4.2 do not need to be repeated.

To run the complete RKNN C Demo, compile the C/C++ source code into an executable, then push the executable, model, test image, and dependency libraries to the board.

4.4.1 Compilation

Take RV1106 running Linux with the armhf architecture as an example. RV1103/RV1103B are also Linux development boards.

Before running `build-linux.sh`, specify the local GCC cross-compiler path. Add the following line at the beginning of `build-linux.sh`:

```
GCC_COMPILER=Projects/arm-rockchip831-linux-uclibcgnueabihf/bin/arm-rockchip831-
linux-uclibcgnueabihf
```

Then compile:

```
cd Projects/rknn_model_zoo

# Usage: ./build-linux.sh -t <target> -a <arch> -d <build_demo_name> [-b
<build_type>] [-m]
# -t : target (rv1103/rv1106); rv1103b/rv1106b and rv1103/rv1106 can share code
other than models
# -a : arch (aarch64/armhf)
# -d : demo name, corresponding to a subdirectory under examples, such as yolov5 or
mobilenet
# -b : build_type(Debug/Release)
# -m : enable address sanitizer; build_type must be set to Debug
./build-linux.sh -t rv1106 -a armhf -d yolov5
```

4.4.2 Push Files to the Board

After compilation, an `install` folder is generated under `rknn_model_zoo`. It contains the executable, dependency libraries, model, and test image. The directory structure is similar to the following:

```
install
└── rv1106_linux_armhf
    └── rknn_yolov5_demo
        ├── lib
        ├── model
        └── rknn_yolov5_demo
```

Push the entire Demo directory to the board:

```
cd Projects/rknn_model_zoo

# For different models and platforms, select the corresponding path under install
adb push install/rv1106_linux_armhf/rknn_yolov5_demo /data/
```

4.5 Run on the Board and Verify the Output

Enter the board and run the executable:

```
adb shell

cd /data/rknn_yolov5_demo/

# Set the dependency library environment
export LD_LIBRARY_PATH=/data/rknn_yolov5_demo/lib

# Usage: ./rknn_yolov5_demo <model_path> <input_path>
./rknn_yolov5_demo model/yolov5s_relu.rknn model/bus.jpg
```

By default, the output image is saved as `rknn_yolov5_demo/out.png`. Run the following command on the computer to pull the result:

```
adb pull /data/rknn_yolov5_demo/out.png .
```

The output image is shown below:

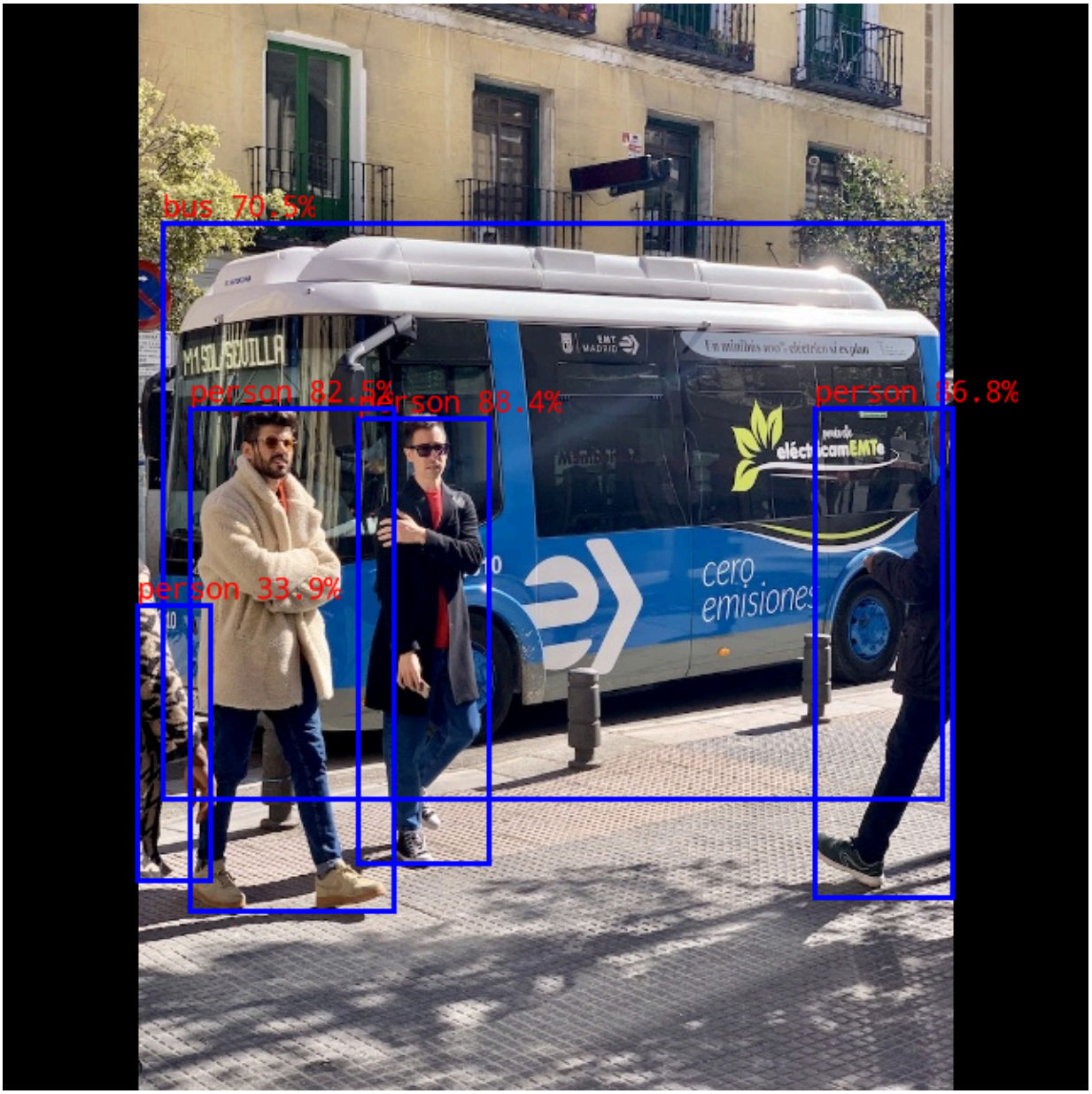

Figure 4-2 RKNN C Demo output image

If `out.png` can be generated and pulled successfully, the first verification of C/C++ compilation, file deployment, Mini Runtime loading, and board-side inference is complete.

4.6 Dataset Accuracy Evaluation (Optional)

The `rknn_model_zoo/datasets` directory is used to store datasets for accuracy evaluation. For YOLOv5, download the COCO validation dataset first.

```
cd Projects/rknn_model_zoo/datasets/COCO

# Download the val2017 dataset
python download_eval_dataset.py
```

To perform dataset accuracy evaluation, install `pycocotools` and specify the evaluation mode and dataset path using `--coco_map_test` and `--img_folder`:

```
pip install pycocotools

cd Projects/rknn_model_zoo/examples/yolov5/python

python yolov5.py \
    --model_path ../model/yolov5s_relu.rknn \
    --target rv1106 \
    --img_folder ../../../datasets/COCO/val2017 \
    --coco_map_test
```

5. Run the RKNN Python Demo in Docker (Optional)

If you do not want to install the RKNN-Toolkit2 environment directly on the computer, you can use a Docker image containing the RKNN-Toolkit2 environment to run the RKNN Python Demo.

> **Note:** The Docker image in this chapter contains only the RKNN-Toolkit2 environment and is mainly used to run the RKNN Python Demo. The board-side RKNPU2 environment still needs to be prepared according to Section 3.3.

5.1 Install Docker

If Docker is already installed, skip this section. Otherwise, refer to the Docker official installation guide: https://docs.docker.com/install/linux/docker-ce/ubuntu/.

After installation, add the current user to the `docker` user group:

```
# Create the docker group
sudo groupadd docker

# Add the current user to the docker group
sudo usermod -aG docker $USER

# Activate the docker group
newgrp docker

# Verify that docker commands can be executed without sudo
docker run hello-world
```

When successful, output similar to the following is displayed:

```
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
719385e32844: Pull complete
Digest: sha256:88ec0acaa3ec199d3b7eaf73588f4518c25f9d34f58ce9a0df68429c5af48e8d
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
```

5.2 Prepare the RKNN-Toolkit2 Docker Image

You can either build an image from a Dockerfile or load a packaged Docker image file.

5.2.1 Build an Image from a Dockerfile

The `docker/docker_file` directory in the RKNN-Toolkit2 project provides Dockerfiles:

```
# xx and x.x.x represent version numbers. Replace them with the actual values
cd Projects/rknn-toolkit2/rknn-toolkit2/docker/docker_file/ubuntu_xx_xx_cpxx

docker build -f Dockerfile_ubuntu_xx_xx_for_cpxx -t rknn-toolkit2:x.x.x-cpxx .
```

5.2.2 Load a Packaged Image

Docker image download link: https://console.zbox.filez.com/l/I00fc3 (Extraction code: `rknn`).

Run:

```
# x.x.x is the RKNN-Toolkit2 version, and cpxx is the Python version

docker load --input rknn-toolkit2-x.x.x-cpxx-docker.tar.gz
```

5.2.3 Check the Image

```
docker images
```

Reference output:

```
REPOSITORY       TAG           IMAGE ID       CREATED       SIZE
rknn-toolkit2    x.x.x-cpxx    xxxxxxxxxxxx   1 hours ago   5.89GB
```

5.3 Run the RKNN Python Demo in the Container

First, prepare RKNN Model Zoo locally according to Section 1.5, and then map the directory and USB device into the container:

```
# -v <host src folder>:<image dst folder> maps a local directory into the container

docker run -t -i --privileged \
    -v /dev/bus/usb:/dev/bus/usb \
    -v "$HOME/Projects/rknn_model_zoo:/rknn_model_zoo" \
    rknn-toolkit2:x.x.x-cpxx \
    /bin/bash
```

After entering the container, prepare the model:

```
cd /rknn_model_zoo/examples/yolov5/model
./download_model.sh
```

Convert the model:

```
cd /rknn_model_zoo/examples/yolov5/python
python convert.py ../model/yolov5s_relu.onnx rv1106 i8 ../model/yolov5s_relu.rknn
```

Run the RKNN Python Demo:

```
python yolov5.py --model_path ../model/yolov5s_relu.rknn --target rv1106
```

After the script runs successfully, the output is similar to the following. `class` is the predicted class, `score` is the confidence score, `(xmin, ymin)` is the upper-left coordinate of the detection box, and `(xmax, ymax)` is the lower-right coordinate.

```
# class @ (xmin, ymin, xmax, ymax) score
person @ (209 243 286 510) 0.880
person @ (479 238 560 527) 0.871
person @ (109 238 231 534) 0.840
bus    @ (91 129 555 464) 0.692
```

6. Common First-Run Issues and Next Steps

This chapter covers only the most common connection, Server, and memory issues encountered during the first run. For more information about tool installation, model conversion, and systematic troubleshooting, refer to *RKNN2_SDK_Development_Guide*. For API-related issues, refer to *RKNN2_Toolkit_Python_API_Reference*, *RKNN2_Toolkit_Lite_Python_API_Reference*, or *RKNN2_Runtime_C_API_Reference* according to the interface being used.

6.1 Device Cannot Be Recognized

If no device is displayed by `adb devices`, check the following in order:

1. Check whether the cable is connected correctly, and reconnect it.
2. Try another USB port on the computer.
3. Replace the data cable.
4. When connecting the development board through USB, make sure that only one ADB Server is running between the local computer and the Docker container. If you need to connect the development board from a Docker container, first run the following command on the local computer:

```
adb kill-server
```

If the following error is displayed, ADB is not installed on the system:

```
command 'adb' not found, but can be installed with:
sudo apt install adb
```

Install it using:

```
sudo apt install adb
```

6.2 `rknn_server` Does Not Start Correctly

The on-board debugging feature of RKNN-Toolkit2 requires the RKNPU2 environment to be installed on the board and `rknn_server` to run correctly. On some development board systems, `rknn_server` is not started automatically at boot. In this case, running the RKNN Python Demo may produce:

```
E init_runtime: The rknn_server on the concected device is abnormal.
```

Start it manually:

```
adb shell
restart_rknn.sh
```

If the problem persists, return to 3.3.2 Check RKNPU2 Mini Runtime and Server to check the components and versions.

6.3 Insufficient Memory Causes Model Loading Failure

RV1103/RV1103B/RV1106/RV1106B development boards have limited memory. If the `rkipc` process is already running on the board, model loading may fail with the following error:

```
E init_runtime: Exception: RKNN init failed. error code: RKNN_ERR_MODEL_INVALID
```

Enter the board and terminate the `rkipc` process:

```
adb shell

# View the currently running processes and find the PID corresponding to rkipc
top

# Terminate the rkipc process
kill -9 PID
```

6.4 Next-Step Documentation

After completing the first run, refer to the corresponding primary document for subsequent development tasks instead of continuing to look for complete development flows, API descriptions, or advanced configuration in this Quick Start.

Search Need / Learning Goal	Recommended Reference
RK3588 and other platforms: First-time RKNN2 SDK environment setup, model conversion, and deployment verification	*RKNN2_SDK_QuickStart*
RK2118 platform: First-time RKNN2 SDK environment setup, model conversion, and deployment verification	*RKNN2_RK2118_QuickStart*
End-to-end development: RKNN2 software stack architecture, model conversion and quantization, accuracy/performance analysis, complete deployment, and troubleshooting	*RKNN2_SDK_Development_Guide*
PC-side model conversion and analysis: RKNN-Toolkit2 Python API methods, parameters, and return values	*RKNN2_Toolkit_Python_API_Reference*
Lightweight board-side Python deployment: RKNN-Toolkit-Lite2 Python API methods, parameters, and return values	*RKNN2_Toolkit_Lite_Python_API_Reference*
Board-side C/C++ deployment: Board-side C APIs, structures, enumerations, error codes, and Mini Runtime related interfaces	*RKNN2_Runtime_C_API_Reference*
Model compatibility evaluation: RKNN supported operator list and related constraints	*RKNN2_Supported_Operators_and_Constraints_Reference*

> **Note:** This document is intended for RV1103/RV1103B/RV1106/RV1106B. The board-side runtime library used throughout this guide is RKNPU2 Mini Runtime (`librknnmrt.so`). *RKNN2_Runtime_C_API_Reference* is the document name for the Runtime C API reference. When using it, select the interfaces applicable to the current platform and SDK version.